

Orbit Wars — v12 Network & Representation

Reference

Observation encoding, the two trunk architectures (legacy ResNet-MLP and the stacked transformer), the actor/critic heads, and the gated-allocation action space

Source of truth: the `orbit_wars_v12/` package

2026-06-19

Abstract

This is the *architecture and representation* reference for the v12 stack: what the policy sees (observation encoding), how it processes it (the two interchangeable trunks), and what it emits (the gated-allocation action and its distributions). Every number here is read directly from the package modules `constants.py`, `config.py`, `env.py`, `policy.py`, and `distributions.py`, and parameter counts are taken from instantiated nets. It documents only what is wired and operational. The separate document `grpo_v12.tex` covers the critic-free RL *update* and a family of variance-reduction tricks — many of which are experimental or have been ruled out — and should *not* be read as a description of the shipped architecture.

Contents

1	Fixed dimensions	2
2	Observation encoding (<code>env_encode</code>)	2
2.1	Per-planet body features (14)	3
2.2	Per-planet threat features (180 = 6×30)	3
2.3	Board globals (10)	4
3	Trunk architectures (<code>policy.py</code>)	4
3.1	Legacy ResNet-MLP trunk (<code>ARCH="trunk"</code> , the default)	4
3.2	Stacked transformer trunk (<code>ARCH="transformer"</code>)	4
4	Heads (shared by both trunks)	5
5	Action space & distributions (<code>distributions.py</code>)	5
5.1	Destination masking	6
5.2	The gate (both variants)	6
5.3	GatedCatDist — categorical WHERE (v7 default, <code>WHERE_DIST="categorical"</code>) . .	6
5.4	GatedAllocDist — Dirichlet WHERE (v6 A/B alternative)	6
6	Parameter counts	6
6.1	Legacy ResNet-MLP trunk	7
6.2	Stacked transformer trunk (8 heads, head dim 64 at width 512)	7

1 Fixed dimensions

These never vary between runs (`constants.py`); they are game/representation physics, not config.

Symbol	Value	Meaning
PLANET_CAP (E)	48	planet/comet slots per env = destination-categorical size
F_DIM (F)	194	per-planet feature width = 14 body + 180 threat
N_BODY_FEATURES	14	per-planet body features (v9: 4-channel seat one-hot ownership)
N_THREAT_FLEETS	30	inbound-fleet slots/planet = N_SOON 15 + N_BIG 15
N_THREAT_FEATS	6	per-fleet descriptor = 4-ch seat one-hot owner + eta + log-ships
G_DIM (G)	10	board-global features
BOARD_SIZE	100	square board; CENTER= 50, SUN_RADIUS= 10

Tensor shapes for a batch of B envs ($B = \text{NUM_GROUPS} \times \text{GROUP_SIZE}$):

Tensor	Shape	Notes
<code>entities</code>	$(B, 48, 194)$	per-planet features, dead slots zeroed
<code>entity_mask</code>	$(B, 48)$	1 = a live planet/comet slot (= <code>p_alive</code>)
<code>action_mask</code>	$(B, 48)$	1 = legal launch source (own, alive, ships > 0)
<code>globals</code>	$(B, 10)$	board summary, ego-relative
<code>action</code>	$(B, 48, 49)$	gated allocation: [:48] WHERE row, [48] fire $\in \{0, 1\}$

2 Observation encoding (`env_encode`)

All features are computed *ego-relative* (relative to the acting seat `ego`), so the same network plays any seat. Ownership is a **4-channel ego-relative seat one-hot** (v9): for body b with owner o , channel $r = (o - \text{ego}) \bmod n_{\text{players}}$ fires; neutral bodies are all-zero. Channel 0 is always “me”; in 2p only channels 0–1 ever fire, so 4p reuses the identical input space.

2.1 Per-planet body features (14)

idx	expr	meaning (normalization)
0	<code>p_x / BOARD_SIZE</code>	x position $\in [0, 1]$
1	<code>p_y / BOARD_SIZE</code>	y position $\in [0, 1]$
2	<code>vmag / THREAT_MAX_SPEED</code>	orbital tangential speed $ \omega \cdot d$ if rotating, else 0 (/6)
3	<code>cw</code>	rotation sense: +1 CCW, -1 CW, 0 if static/comet
4	<code>p_radius / 3</code>	planet radius
5	<code>p_prod / 5</code>	production rate
6	<code>ship_log_t(p_ships)</code>	garrison, log-compressed $\log_{1p}(n) / \log 1000$
7–10	seat one-hot $[b_{7a..d}]$	ownership [self, enemy+1, enemy+2, enemy+3]; neutral all-0
11	<code>dist / DIAG_HALF</code>	distance from sun/centre (/ half-diagonal)
12	<code>comet</code>	1 if this slot is a comet
13	<code>actable</code>	1 if own $\wedge$ alive $\wedge$ ships > 0 (= <code>action_mask</code>)

2.2 Per-planet threat features (180 = 6 × 30)

For each planet slot, among all in-flight fleets *currently targeting it* (analytic aim from `fleet_target_batch`), select the `N_SOON`= 15 smallest-ETA fleets and the `N_BIG`= 15 largest fleets (size key $|\text{ships}| \cdot 2^{20}$ –seq for a stable tie-break), giving up to 30 inbound-fleet descriptors; missing slots are zero-padded. Each descriptor is 6 features:

- **owner one-hot (4)** — an ego-relative seat one-hot of the fleet’s owner, $\text{rel} = (\text{f_owner} - \text{ego}) \bmod n_{\text{players}}$: channel 0 = self, channels 1–3 = the opponents in seat order (empty slots all-zero). Mirrors the body ownership one-hot, so the policy can tell *which* opponent launched an incoming fleet.
- **etf** — `ETA/THREAT_ETA_SCALE` (/100);
- **slg** — `ship_log_t(|ships|)`, log-compressed fleet size.

Stacked as $6 \times 30 = 180$. Final `entities` = [body || threat] is multiplied by `p_alive` so dead slots are 0.

v12 change (per-opponent threat identity). Before v12 the owner was a single *sign* (+1 self / -1 any-opponent), so in 4p all three opponents collapsed into one value and the policy could not attribute an incoming fleet to a specific rival. The 4-channel one-hot fixes this (2p still only ever lights channels 0–1, so the 2p input is unchanged in meaning); it raises `F_DIM` from 104 to 194. Members trained on the old 104-feature obs still play via `checkpoint.LegacyObsAdapter`, which collapses the one-hot back to the sign at forward time ($\text{sign} = \text{ch}_0 - \sum_{k=1}^3 \text{ch}_k$).

2.3 Board globals (10)

idx	expr	meaning
0	<code>step_ct / T</code>	game progress (time fraction)
1	<code>ang_vel * 10</code>	board rotation angular velocity
2	<code>ship_log_t(my_ships)</code>	my total garrison (log)
3	<code>ship_log_t(en_ships)</code>	all-opponents garrison, pooled (log)
4	<code>my_pl / total</code>	fraction of live bodies I own
5	<code>en_pl / total</code>	fraction opponents own
6	<code>neu_pl / total</code>	fraction neutral
7	<code>ship_log_t(my_fleet)</code>	my in-flight fleet ships (log)
8	<code>ship_log_t(en_fleet)</code>	opponents' in-flight fleet ships (log)
9	<code>min(npl,40)/40</code>	live-body count, scaled

The per-planet ownership is a one-hot *per opponent*, but the global enemy aggregates (idx 3, 5, 8) pool *all* opponents (an everyone-vs-me summary).

3 Trunk architectures (policy.py)

The trunk maps **entities** (B, E, F) to per-planet feature tokens (B, E, h) with $h = \text{HIDDEN}$. `cfg.ARCH` selects one of two interchangeable trunks; everything downstream (heads, action, checkpoint format) is identical. The activation in every ResNet-MLP block is `cfg.ACT_FN` (default `tanh`).

3.1 Legacy ResNet-MLP trunk (ARCH="trunk", the default)

```

entities (B,E,104)
-> proj: Linear(104 -> 512)
-> [optional] cross-planet single-head self-attention    (USE_ATTENTION, default OFF)
      pre-LN: x = x + attn_o( softmax(QK^T/sqrt h, dead-key masked) V )
-> [optional] GLU:  x = x + glu_out( glu_val(x) * sigmoid(glu_gate(x)) )    (USE_GLU, ON)
-> N_RES_BLOCKS x pre-LN ResNet-MLP block:
      x = x + res_b[i]( act( res_a[i]( LN(x) ) ) )      (N_RES_BLOCKS = 86)
-> ln_out: LayerNorm(512)

```

Block used: pre-LayerNorm residual MLP (two $h \times h$ linears with a pointwise activation between, added back as a skip). At the default width/depth this is the “**512×86**” trunk. With `GRAD_CHECKPOINT` (default *on*) the residual blocks are activation-checkpointed (recomputed in the backward pass) so the deep stack fits in VRAM.

Important: `USE_ATTENTION` defaults to *False*, so at the shipped defaults the legacy trunk has *no cross-planet mixing inside the trunk* — each planet token is processed independently; planets only interact later, at the masked-mean value pool and the bilinear WHERE head. The transformer trunk (below) restores per-layer cross-planet attention.

3.2 Stacked transformer trunk (ARCH="transformer")

```

entities (B,E,104)
-> proj: Linear(104 -> 512)

```

```

-> stem:    N_STEM_RES x pre-LN ResNet-MLP block    (=1, blocks PRE-attention)
-> stack:   N_TX_LAYERS x TxEncoderLayer            (=6)
-> head:    N_HEAD_RES x pre-LN ResNet-MLP block    (=4, blocks POST-attention)
-> ln_out:  LayerNorm(512)

```

So the "pre / post attention" blocks the question asks about are the **stem** (N_STEM_RES , before the encoder stack) and the **head** (N_HEAD_RES , after it) — both are the same pre-LN ResNet-MLP block as the legacy trunk. The attention itself lives in N_TX_LAYERS encoder layers.

One TxEncoderLayer (pre-LN encoder block).

$$x \leftarrow x + \text{MHA}(\text{LN}_1(x)), \quad x \leftarrow x + \text{MLP}(\text{LN}_2(x)).$$

- **Multi-head attention:** N_HEADS heads (head dim = h/N_HEADS), separate **q,k,v,o** linears, computed with `F.scaled_dot_product_attention` (flash / mem-efficient). Dead-planet *keys* are masked with an additive -10^9 , matching the legacy single-head mask.
- **MLP:** `mlp_in`: Linear($h \rightarrow \text{TX_MLP_RATIO} \cdot h$), activation, `mlp_out`: Linear back to h . Default ratio 4 $\Rightarrow$ hidden 2048.

Heads-divide-width footgun. N_HEADS must divide `HIDDEN` (`assert h % n_heads == 0`). The config *defaults* pair `HIDDEN= 512` with $N_HEADS= 12$, which is *invalid* ($512/12$ is not integer); building a 512-wide transformer requires overriding N_HEADS (e.g. 8, head dim 64 — the value `INVITE_CFG` uses). The numbers below use 8 heads at width 512.

4 Heads (shared by both trunks)

Trunk tokens $\text{tok} \in \mathbb{R}^{B \times E \times h}$ and a board-globals embedding $\text{gp} = \text{relu}(\text{g_embed}(\text{globals})) \in \mathbb{R}^{B \times d_g}$ ($d_g = D_G = 32$) feed all heads; per planet the head input is $hh = [\text{tok} \parallel \text{gp}] \in \mathbb{R}^{h+d_g}$ ($= 544$).

- **WHERE head** (`DEST_HEAD`, default **bilinear**): pointer scores $\text{dest_logits}[b, s, d] = \langle \text{dest_q}(hh_s), \text{dest_k}_d \rangle$ a (B, E, E) source $\times$ destination matrix. (**linear** alternative: one Linear($544 \rightarrow E$)).
- **GATE head:** `gate_head` Linear($544 \rightarrow 1$) $\Rightarrow$ per-source launch logit (B, E) .
- **VALUE head (critic):** masked-mean pool of `tok` over live planets, concat `gp`, then `val_in` Linear($544 \rightarrow h$) $\rightarrow$ `VALUE_RES_BLOCKS` pre-LN ResNet-MLP blocks ($= 4$) $\rightarrow$ `val_out` Linear($h \rightarrow 1$). Targets are normalized by **PopArt** (output-preserving running mean/var; the head predicts normalized values, raw $V = \text{denorm}$).
- **AUX reward head** (optional, `AUX_REWARD_PRED`, default off): a shallow `rew_in` $\rightarrow$ `act` $\rightarrow$ `rew_out` off the *same* pooled features, regressing the per-step reward r_t as a representation auxiliary. Built only when enabled, so checkpoints are byte-identical when off.

5 Action space & distributions (distributions.py)

The actor is a **gated per-source allocation**. For every owned source planet it emits a **WHERE** distribution over destinations and a launch **GATE**. *On fire, the planet dispatches its full garrison routed by the WHERE row; on no-fire it holds.* The action bundle is $(B, E, E+1)$: $[:E]$ is the allocation row, $[E]$ is fire $\in \{0, 1\}$. Log-prob and entropy *sum over owned sources* (the **WHERE** term only counts when firing).

5.1 Destination masking

WHERE logits are clamped to $\pm \text{LOGIT_CLAMP}$ ($= 8$) and then masked to -10^9 on: the diagonal (self), dead destinations ($\text{alive} < 0.5$), and sun-blocked destinations ($\text{reach} < 0.5$, a straight shot absorbed by the sun, recovered geometrically in both rollout and update). So the WHERE support is exactly the legal, reachable targets.

5.2 The gate (both variants)

$g = \sigma(\text{gate_logit})$. **Train:** $p = g$ (smooth, full-support gradient — the deadzone clamp used to zero it and vanish the gate gradient). **Deploy:** hard deadzone $p = \text{clip}((g - \text{GATE_TRIM_LO}) / (\text{GATE_TRIM_HI} - \text{GATE_TRIM_LO}), 0, 1)$. Then $p \leftarrow \text{clip}(p, \text{GATE_EPS}, 1 - \text{GATE_EPS})$ and $\text{fire} \sim \text{Bernoulli}(p)$. The gate is a *logistic* (sigmoid), not a softmax.

5.3 GatedCatDist — categorical WHERE (v7 default, WHERE_DIST="categorical")

This is where the per-source **softmax** lives. After masking,

$$\text{ls} = \log \text{softmax}(\text{glogits}, \text{dim} = -1) \in \mathbb{R}^{B \times E \times E},$$

a log-probability over destinations *for each source planet*. Then:

- **sample:** $\text{dest} \sim \text{Categorical}(\text{ls})$; $\text{fire} \sim \text{Bernoulli}(p)$. (With **GRPO_CRN** the WHERE Gumbel-max noise and the gate uniform are shared across a world-group — common random numbers.)
- **log-prob (per source):** $\ell = \underbrace{\text{fire} \log p + (1 - \text{fire}) \log(1 - p)}_{\text{gate}} + \text{fire} \cdot \text{ls}[\text{dest}]$, summed over owned sources. (Optionally the WHERE term is Rao-Blackwellized over the gate Bernoulli, **GRPO_RB_GATE**.)
- **entropy:** Bernoulli gate entropy + $\text{ENT_DIR_COEF} \cdot p \cdot H_{\text{cat}}$, where $H_{\text{cat}} \leq \ln E$ is bounded. **ENT_DIR_COEF** defaults to 0, so the bonus is *gate-only* (the categorical entropy would blur a BC-sharpened WHERE head).
- **greedy / deploy:** $\text{dest} = \arg \max \text{glogits}$; fire sampled at the trained rate.

The bounded-sensitivity log softmax log-prob is what makes PPO-after-BC stable.

5.4 GatedAllocDist — Dirichlet WHERE (v6 A/B alternative)

$\alpha = \text{softplus}(\text{logits}) \cdot \text{ALLOC_KAPPA} + 10^{-3}$, $\text{rows} \sim \text{Dirichlet}(\alpha)$; the full garrison is split across destinations by the sampled simplex. Its greedy decode uses a *temperature-sharpened* softmax ($\tau = \text{GREEDY_TAU} = 0.5$) over the masked logits, because the Dirichlet mean spreads the garrison too thin to land a concentrated strike.

6 Parameter counts

From instantiated nets (fp32). “trunk” = representation only; “heads” = $\text{g_embed} + \text{WHERE} + \text{gate} + \text{value}$ (aux head off). $E = 48$, $F = 104$, $G = 10$, $d_g = 32$, **VALUE_RES_BLOCKS** = 4 throughout.

6.1 Legacy ResNet-MLP trunk

config	trunk	heads	total	fp32 / fp16
512×86 (default)	46.16 M	2.94 M	49.10 M	196 / 98 MB
512×32	17.73 M	2.94 M	20.68 M	83 / 41 MB
256×32	4.64 M	0.59 M	5.23 M	21 / 10 MB

Component breakdown of the default **512×86**:

component	params
proj (194 → 512)	99,840
GLU (glu_gate+glu_val+glu_out)	787,968
86 ResNet-MLP blocks (res_a+res_b)	45,176,832
ln_res+ln_out	89,088
g_embed (10 → 32)	352
WHERE (dest_q+dest_k)	558,080
GATE	545
VALUE (in + 4 res + ln + out)	2,384,897
total	49,097,602

6.2 Stacked transformer trunk (8 heads, head dim 64 at width 512)

config	trunk	heads	total	fp32 / fp16
512h / 6L / 8-head (current cfg, heads fixed)	21.65 M	2.94 M	24.59 M	98 / 49 MB
512h / 12L / 8-head (it426-class)	40.57 M	2.94 M	43.51 M	174 / 87 MB
256h / 4L / 8-head (INVITE_CFG canonical)	3.62 M	0.34 M	3.96 M	16 / 8 MB

Component breakdown of **512h / 6L / 8-head** (per-encoder-layer = 3,152,384):

component	params
proj (194 → 512)	99,840
stem (N_STEM_RES= 1 pre-attn ResNet-MLP)	526,336
6× MHA (q,k,v,o)	6,303,744
6× MLP (mlp_in+mlp_out, ratio 4)	12,598,272
6× LayerNorm (LN ₁ ,LN ₂)	12,288
head (N_HEAD_RES= 4 post-attn ResNet-MLP)	2,105,344
ln_out	1,024
heads (g_embed + WHERE + gate + value)	2,943,874
total	24,590,722

Submission size ceiling. The Kaggle 100 MB cap implies ≤ 26 M params at fp32; the default 512×86 (49 M) and the 512h/12L transformer (43 M) only fit at fp16. The 512×32 trunk and the 512h/6L transformer are the largest fp32-shippable configs.

7 Config-knob map

The architecture is fully determined by these `Config` fields (`config.py`); defaults shown.

field	default	controls
ARCH	"trunk"	legacy ResNet-MLP vs "transformer"
HIDDEN	512	trunk width h
N_RES_BLOCKS	86	[trunk] residual MLP blocks
USE_GLU	True	[trunk] GLU mixing block
USE_ATTENTION	False	[trunk] cross-planet self-attention
N_TX_LAYERS	6	[tx] encoder layers
N_HEADS	12	[tx] attention heads (must divide HIDDEN!)
TX_MLP_RATIO	4	[tx] per-layer MLP expansion
N_STEM_RES	1	[tx] ResNet-MLP blocks <i>before</i> the stack
N_HEAD_RES	4	[tx] ResNet-MLP blocks <i>after</i> the stack
VALUE_RES_BLOCKS	4	critic-head residual blocks
D_G	32	board-globals embedding dim
ACT_FN	"tanh"	ResNet-MLP activation
DEST_HEAD	"bilinear"	WHERE head: pointer vs linear
WHERE_DIST	"categorical"	actor: <code>GatedCatDist</code> vs Dirichlet
USE_POPART	True	value-target normalization
GRAD_CHECKPOINT	True	activation-checkpoint trunk blocks